

## Intro:

Period: 7

Group Members: Gordon Doan, Kevin Lin, Jophy Wang

Group Name: Rhythm Rickles

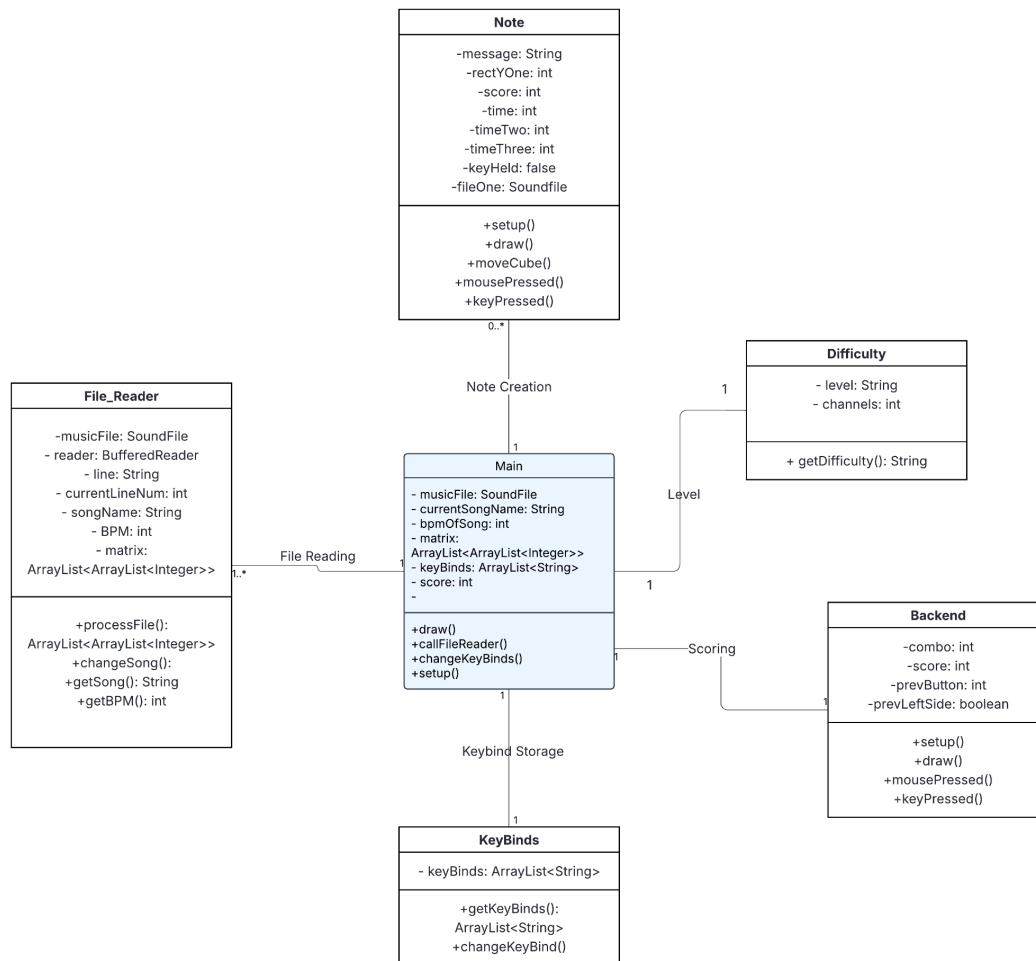
Project Title: Osu Mania Dup

## Description:

We are creating a rhythm game with four channels/lanes where incoming notes need to be pressed at a precise time using the arrow keys (can be reprogrammed). This program will be similar to games such as osu!mania and Friday Night Funkin'. Music will be played in the background, and there will be a score kept based on how well you press incoming keys. The higher your score, the better you do. There will be three songs to select, each with a different beatmap. There will also be an option to change keybinds. To implement this, we will need to use the and PVector libraries. We will add a map editor, allowing users to create their own beat maps and then play them. We will use createWriter() to save the file, allowing the user to edit the beat map and then play it.

## UML Diagram

**SUBJECT TO CHANGE (MORE CLASSES WILL BE ADDED)**



## How does it work?

Select your desired keybinds on the menu. The default keybinds will be the arrow keys and WASD. You will also be able to change the colors of your keys by pressing [placeholder], which will bring up a menu. Then, select a song to play. You can return to the selection menu by pressing [placeholder]. To play the game, you must line up incoming notes with set notes at the bottom of your screen by pressing the corresponding keybind that you've selected. The closer you are to perfectly lining the keys up, the more points you gain. There will also be longer keys, where you must hold down the key until it ends. You will also have to time the "letting go" part. There is also a combo counter that counts how many notes you've hit in a row. After the song is over, you will be presented with your final score and an option to either play again or play another song.

## Functionalities / Issues

### Functionalities Finished/Operational

- FileReader returning a 2D array with channel & time values
- Part of the main class where key pressing is controlled
- Backend development (Jophy's Portion)
- Drawing the channels
- Animation of notes coming down the channels

### Functionalities Planned To Be Done

- Difficulty levels/Map Editor (Kevin is working on this)
- BPM tinkering
- Note pressing tolerance
- Combo display

### Issues (So Far...)

- Problem with Keycode vs Key values for reassigning keybinds (**Resolved**)
- How to implement whole notes... press to hold (Resolved... in terms of having an idea on how to fix it)
- Encountered an issue with FPS limiting the BPM when going beyond 120fps due to hardware limitations (We will likely set the FPS to 60 to avoid this problem and use doubles to ensure this is not an issue)

## Log:

### Gordon:

- Working on Keybinds and drawing (Keybinds are done)
- FPS tinkering
- Channel drawing (Done)
- Note drawing (Done)

Jophy:

- Music Implementation- In Progress
- Song Selections- In Progress
- Combo Counter- In Progress
- Scoring Table + Scoring Grades- In Progress
- Scoring Tolerances- In Progress
- Hold Key Timing- In Progress
- All of the above require Gordon to implement

Kevin:

- FileReader (Finished)
- The main class receives data from the array (Need Gordon)
- Working with Gordon on FPS tinkering
- Editor (Working on)

Lane, time, note type

#### osu!mania PROCESSING

- We have a text file with an array that is read by a method in the main class

#### Specs

- 4 Channels / Lanes
- Color-coded assets

#### Task:

- Implement BPM of Track
- Implement difficulties
- Implement songs + music files
- Implement beatmap text files
- Implement keybinds
- Implement settings to swap keybinds
- Implement how well you pressed the keys
- Implement reading beatmaps
- Implement start + end screen
- Implement hold keys

#### Data structure:

- ArrayList
- 2d Array

- {{0,0,0,0},
- {0,0,0,0},
- {0,0,0,0},
- {0,0,0,0},
- }

Libraries Required:

Sound - The Processing Foundation

PVector - The Processing Foundation probably

evil mania

eviler mania

Text file format:

<song name>

<BPM>

<music file>

<channel, time>

<channel, time>

Etc...